

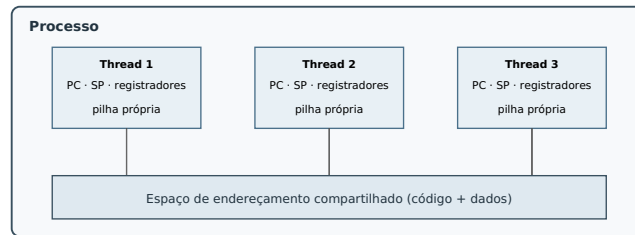
# Aula 06 — Processos (parte 1): Threads

Resumo 7 de 14 · arquivo aula06 · 25 imagens / 18 slides · Cap. 3 de van Steen & Tanenbaum

Começa o tema **Processos**, e esta primeira parte é toda sobre **threads**. A ideia central: threads são “processadores virtuais em software” que permitem executar várias linhas de instrução dentro de um mesmo processo, compartilhando memória. Isso é o que torna clientes e servidores eficientes em sistemas distribuídos.

## 1. Processador, thread e processo

Três definições encaixadas: um **processador** executa automaticamente uma série de instruções; uma **thread** é a menor unidade de software em que um conjunto de instruções roda num dado contexto; um **processo** é a unidade em cujo contexto uma ou mais threads executam. Salvar o contexto de uma thread = parar a execução e guardar tudo que é preciso para retomá-la depois.



Um processo contém várias threads: cada thread tem seu próprio PC, registradores e pilha, mas todas **compartilham** o mesmo espaço de endereçamento.

### Os três contextos

- **Contexto do processador:** o mínimo guardado nos registradores para executar instruções (ponteiro de pilha, contador de programa, registradores de endereçamento).
- **Contexto da thread:** o contexto do processador + o estado da thread.
- **Contexto do processo:** o contexto da thread + pelo menos os registradores da MMU (mapeamento de memória).

Consequências importantes: as threads **compartilham o espaço de endereçamento**, então a troca de contexto entre threads pode ser feita até sem envolver o SO; já a **troca de processo é mais cara** (interrompe para o kernel, atualiza mapas de memória, invalida cache). Criar/destruir threads é muito mais barato que criar/destruir processos.

## 2. Por que usar threads

- **Evitar bloqueios desnecessários:** num processo de thread única, uma operação de E/S bloqueia tudo; com várias threads, o SO passa a CPU a outra thread do mesmo processo.
- **Explorar paralelismo:** threads podem rodar de fato em paralelo em CPUs multinúcleo.
- **Evitar trocas de processo** (caras) e **organizar melhor o software**.

**Trade-off:** como as threads usam o mesmo espaço de endereço, são mais *propensas a erros* — não há proteção de memória entre elas — em troca de trocas de contexto mais rápidas.

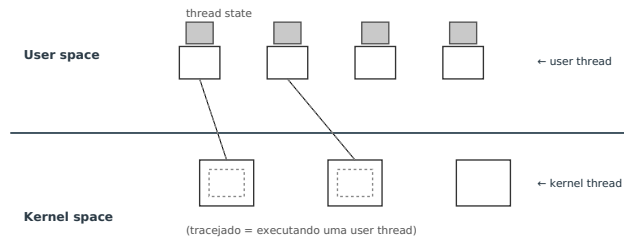
### O custo de uma troca de contexto

Há **custos diretos** (a troca em si e o código que a trata) e **custos indiretos**, principalmente por “bagunçar a cache”: depois de trocar, a thread nova acessa outros dados e desaloja os blocos da anterior, que precisará recarregá-los quando voltar.

## 3. Threads no SO: usuário × kernel

A pergunta central: as threads devem ser implementadas pelo kernel do SO ou em nível de usuário?

- **Nível de usuário:** tudo é tratado dentro do processo → **muito eficiente**. Porém, se o kernel bloquear uma thread, **o processo inteiro bloqueia** (o kernel nem distingue as threads), o que atrapalha o tratamento de eventos externos.
- **Nível de kernel:** o kernel implementa as threads e toda operação (criar, destruir, sincronizar) vira **chamada de sistema**. Bloqueio deixa de ser problema (o kernel agenda outra thread do processo) e eventos externos são fáceis de tratar. O custo é **menos eficiência** (cada operação exige ir ao kernel).
- **Combinada (dois níveis, N:M):** threads de kernel que executam threads de usuário, tentando unir eficiência e flexibilidade.



Modelo combinado N:M: várias threads de usuário são mapeadas sobre um número menor de threads de kernel; uma thread de kernel “tracejada” está no momento executando uma thread de usuário.

## 4. Threads no cliente

No **cliente web multithread**, as threads escondem a latência da rede: o navegador vê que a página precisa de vários arquivos e busca cada um numa thread separada (cada uma faz um HTTP bloqueante), exibindo-os conforme chegam. Da mesma forma, um cliente pode fazer várias chamadas **RPC** em paralelo, uma por thread, e esperar todas — se forem para servidores diferentes, há ganho de velocidade quase linear.

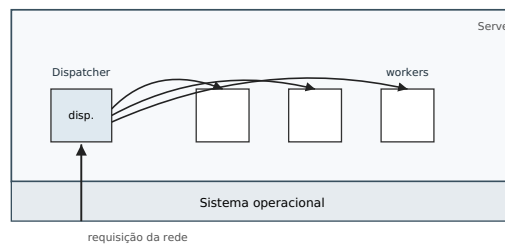
Mas ajuda mesmo? Mede-se pelo **paralelismo de nível de thread (TLP)**, onde  $c_i$  é a fração do tempo com exatamente  $i$  threads ativas:

$$TLP = ( \sum_{i=1..N} i \cdot c_i ) / ( 1 - c_0 )$$

Na prática, um navegador típico tem TLP entre **1,5 e 2,5** — ou seja, as threads servem mais para **organizar logicamente** o navegador do que para ganhar muito paralelismo.

## 5. Threads no servidor

No servidor, threads melhoram o **desempenho** (iniciar thread é mais barato que iniciar processo; um servidor de thread única desperdiça um multiprocessador; dá para atender a próxima requisição enquanto responde a anterior) e a **estrutura** (chamadas bloqueantes simples deixam o código menor e mais fácil de entender). A organização clássica é o modelo **dispatcher/worker**:



Modelo dispatcher/worker: uma thread despachante recebe a requisição da rede e a distribui para uma das threads operárias (workers).

Comparando as três formas de organizar um servidor:

| Modelo                   | Características                               |
|--------------------------|-----------------------------------------------|
| Multithreading           | Paralelismo + chamadas de sistema bloqueantes |
| Processo de única thread | Sem paralelismo + chamadas bloqueantes        |
| Máquina de estado finito | Paralelismo + chamadas não bloqueantes        |